

Пререквизиты

1. Необходимо установить редактора кода IntelliJ IDEA.

Для этого переходим [по ссылке](#) (рис. 1), скачиваем установочный файл, распаковываем и следуем инструкциям по установке.

Примечание: скачивайте **community edition**, поскольку это бесплатная версия и содержит нужный функционал (**javafx**)

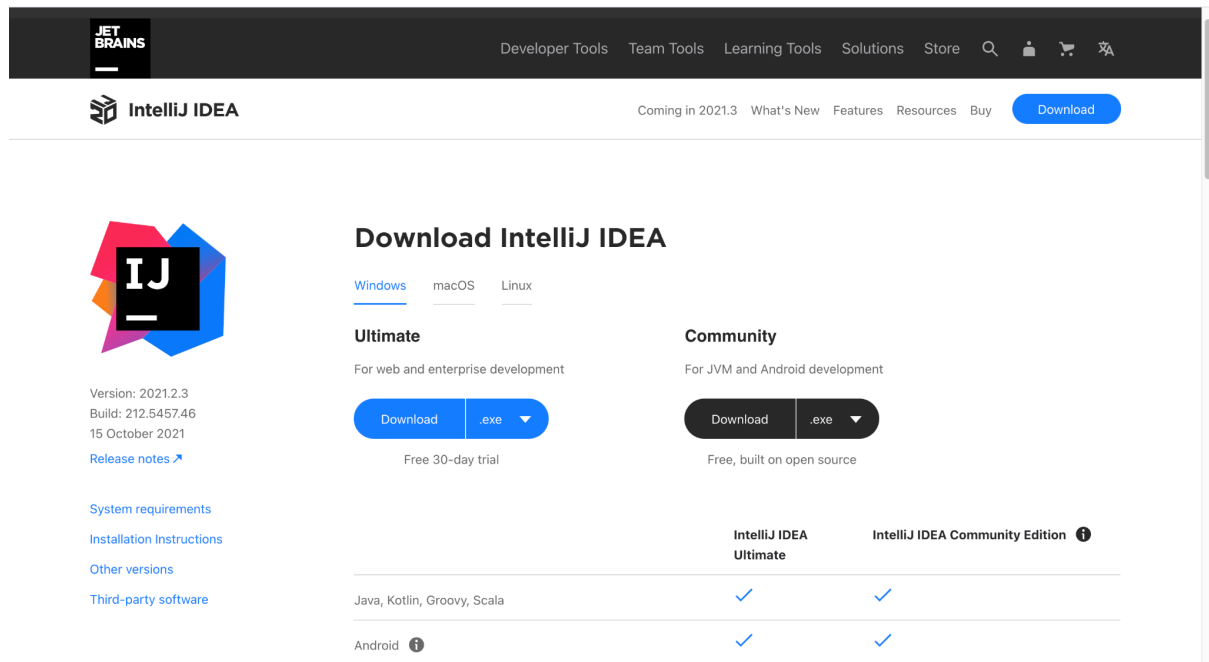


Рис.1 Страница установщика IntelliJ IDEA

2. Необходимо установить Scene Builder (GUI редактор)

Установочной файл можно найти по [ссылке](#). Страница загрузка выглядит, как на рис. 2. После скачивания установите программа следуя подсказкам.

 **GLUON**



and Gluon offerings including and registration allows for team are available, including training Gluon Mobile, Gluon Desktop, members to quickly and easily and custom consultancy and Gluon CloudLink. focus on their specific layer of services. application development.

Download Scene Builder

Scene Builder **17.0.0** was released on **Sep 29, 2021**.

*You can use this Scene Builder version together with **Java 11 and higher**.*

Product	Platform	Download
Scene Builder	Windows Installer	Download
Scene Builder	Mac OS X dmg	Download
Scene Builder	Linux RPM	Download
Scene Builder	Linux Deb	Download
Scene Builder Kit info	Jar File	Download

License: Scene Builder 17 is licensed under the BSD license.

Download Scene Builder for Java 8

Scene Builder **8.5.0** is for users who are still on Java 8. It was released on **Jun 5, 2018**.

Рис.2 Страница установщика Scene Builder

3. Для многих лабораторных работ вам понадобится СУБД. В данном пособии будет показан пример работы с СУБД MySQL.

Сначала скачаем сервер, расположенный по [ссылке](#). На рис.3 показано окно с установщиками. Скачайте и установите программу, следуя инструкциям.

General Availability (GA) ReleasesArchives

MySQL Community Server 8.0.26

Select Operating System:

Looking for previous GA versions?

Microsoft Windows

Recommended Download:

MySQL Installer for Windows

All MySQL Products. For All Windows Platforms. In One Package.

Starting with MySQL 5.6 the MySQL Installer package replaces the standalone MSI packages.

Windows (x86, 32 & 64-bit), MySQL Installer MSI



[Go to Download Page >](#)

Other Downloads:

Windows (x86, 64-bit), ZIP Archive (mysql-8.0.26-winx64.zip)	8.0.26	208.2M	Download
		MD5: db32c0669cc809abb465bb56e76c77a1	Signature
Windows (x86, 64-bit), ZIP Archive Debug Binaries & Test Suite (mysql-8.0.26-winx64-debug-test.zip)	8.0.26	500.2M	Download
		MD5: 9226e8c02fec0671ca40a31bfee7fff1	Signature

 We suggest that you use the [MD5 checksums](#) and [GnuPG signatures](#) to

Рис. 3 Окно страницы с установщиками Mysql Server

4. Необходимо скачать GUI инструмент для работы с БД MySQL - **MySQL Workbench**. Он расположен по [адресу](#). Страница с установщиком показано на рис. 4.

Примечание: Следите за версиями MySQL Server и Workbench. Версия сервера **не должна быть выше**, чем версия Workbench, поскольку в таком случае высока вероятность того, сервер и GUI viewer не будут работать вместе. Например, сервер может иметь версию 8.26, а Workbench - версию 8.27, но не наоборот.

[General Availability \(GA\) Releases](#)[Archives](#)

MySQL Workbench 8.0.27

Select Operating System:

Microsoft Windows

Recommended Download:

MySQL Installer for Windows

All MySQL Products. For All Windows Platforms. In One Package.

Starting with MySQL 5.6 the MySQL Installer package replaces the standalone MSI packages.

Windows (x86, 32 & 64-bit), MySQL Installer MSI

[Go to Download Page >](#)

Other Downloads:

Windows (x86, 64-bit), MSI Installer	8.0.27	42.6M	Download
(mysql-workbench-community-8.0.27-winx64.msi)	MD5: 36949cdb19e4e9f54da6dd4ea8196a73 Signature		

We suggest that you use the [MD5 checksums](#) and [GnuPG signatures](#) to verify the integrity of the packages you download.

Рис. 4 Страница с установщиком Mysql Workbench

5. Mysql connector - java библиотека, используемая для подключения к mysql серверу. Необходимо скачать ее по этому [адресу](#). На рис. 5 показана страница с пакетами, необходимо нажать по ссылке. Далее вы будете перенаправлены на новую страницу, на которой нужно выбрать “No, thanks, just start my download” (рис. 6), и загрузка начнется автоматически. Далее распакуйте архив. В результате вы получите папку, как на рис. 7. В дальнейшем эту папку необходимо будет положить в каждый проект для каждой лабораторной работы.

MySQL Community Downloads

Connector/J

General Availability (GA) Releases Archives

Connector/J 8.0.27

Select Operating System:

Platform Independent

Platform Independent (Architecture Independent), Compressed TAR Archive (mysql-connector-java-8.0.27.tar.gz)	8.0.27	4.0M	Download
Platform Independent (Architecture Independent), ZIP Archive (mysql-connector-java-8.0.27.zip)	8.0.27	4.8M	Download

MD5: 9243dc26efa3909b13517e002a89d7f5 | Signature

MD5: 3d054e96f5b70537cf53c4dd31f11eca | Signature

We suggest that you use the MD5 checksums and GnuPG signatures to verify the integrity of the packages you download.

ORACLE © 2021, Oracle Corporation and/or its affiliates

[Legal Policies](#) | [Your Privacy Rights](#) | [Terms of Use](#) | [Trademark Policy](#) | [Contributor Agreement](#) | [Настройки cookie](#)

Рис. 5 Страница с библиотекой для подключения к mysql server

MySQL Community Downloads

Login Now or Sign Up for a free account.

An Oracle Web Account provides you with the following advantages:

- Fast access to MySQL software downloads
- Download technical White Papers and Presentations
- Post messages in the MySQL Discussion Forums
- Report and track bugs in the MySQL bug system

Login »

using my Oracle Web account

Sign Up »

for an Oracle Web account

MySQL.com is using Oracle SSO for authentication. If you already have an Oracle Web account, click the Login link. Otherwise, you can signup for a free account by clicking the Sign Up link and following the instructions.

No thanks, just start my download.

ORACLE

© 2021, Oracle Corporation and/or its affiliates

[Legal Policies](#) | [Your Privacy Rights](#) | [Terms of Use](#) | [Trademark Policy](#) | [Contributor Agreement](#) | [Настройки cookie](#)

Рис. 6 Страница редиректа после нажатия на библиотеку

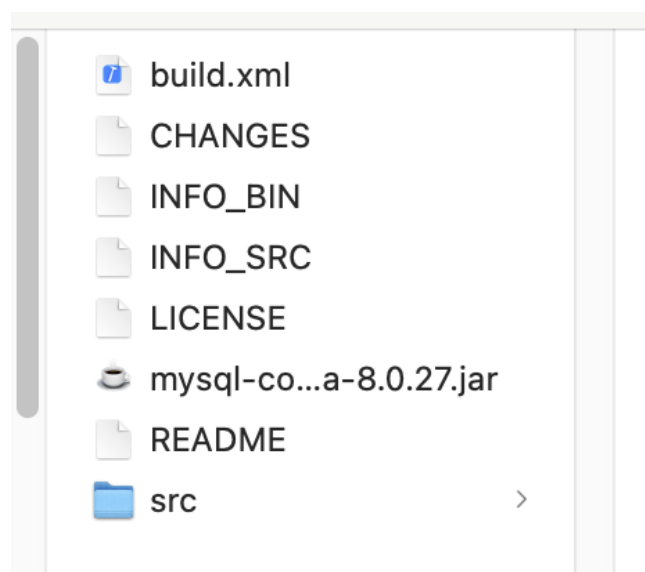


Рис. 7 Распакованный mysql connector

7. javafx-swing.jar – библиотека для создания графических интерфейсов в Java-приложениях. Необходимо скачать ее по этому [адресу](#).



Рис. 8. Библиотека javafx-swing.jar

Лабораторная работа 1. Зарплаты

1. Создаем новый проект, как на рис. 9. Даем ему имя “laboratoryWorks”. Если у вас не выбран “JDK”, то проверьте или докачайте его из выпадающего меню (подходящие версии +16), как на рис. 9. Нажмите “Next”.

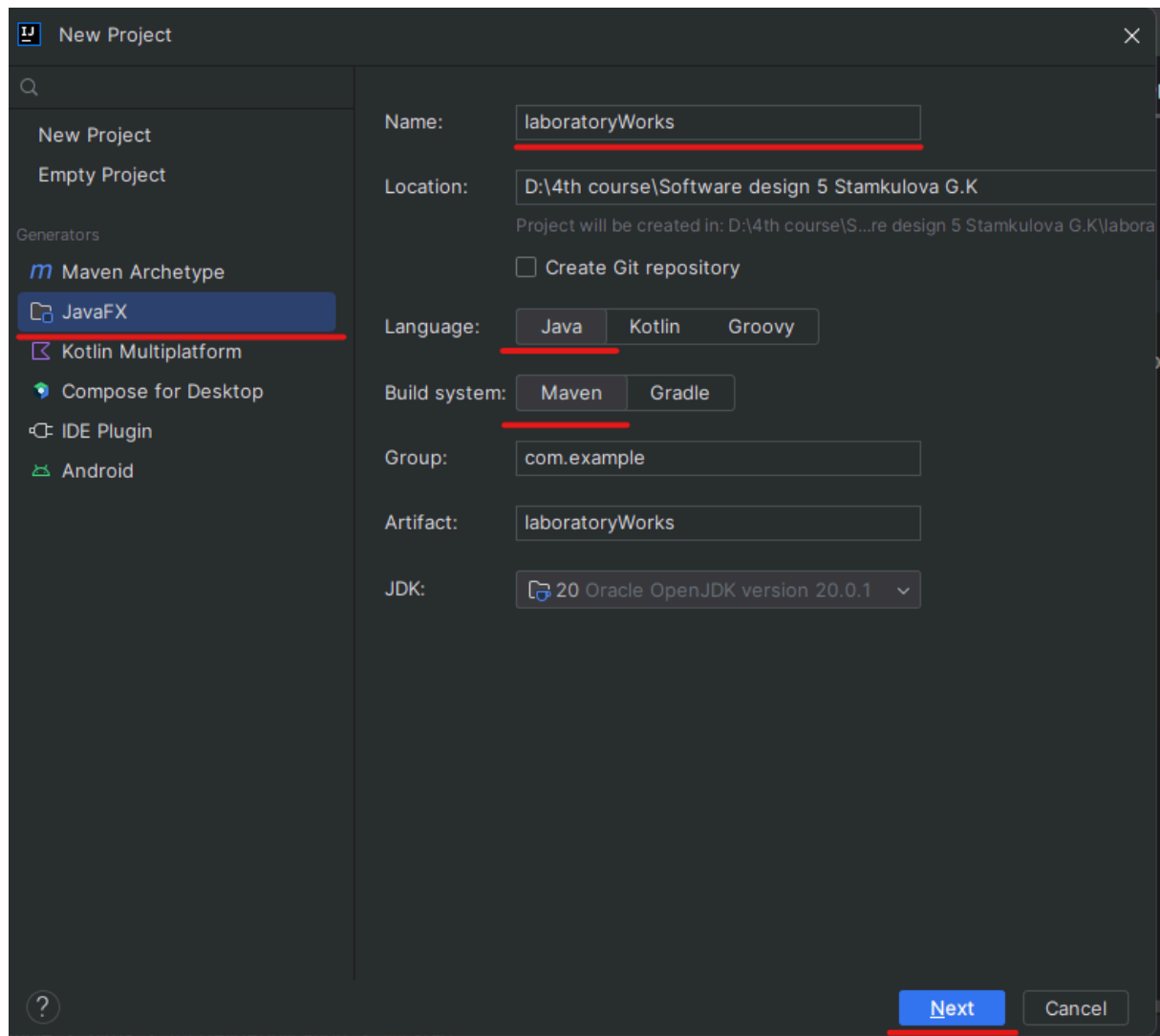


Рис. 9 Стартовое окно создания проекта

Далее выберите все необходимые зависимости, нажмите “Далее”

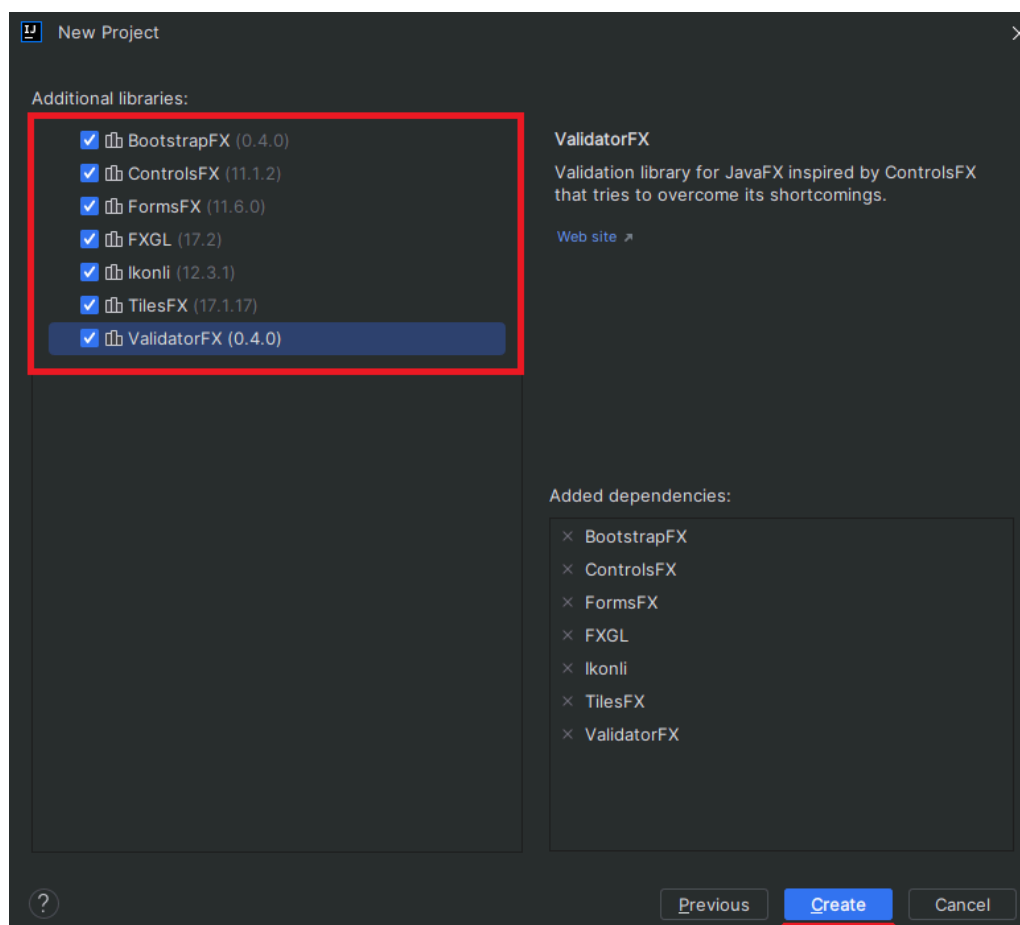


Рис. 10 Выбор зависимостей

Структура проекта показана на рисунке 11. **MainController** это файл, в котором прописываются элементы графического интерфейса и обработчики событий, в котором инициализируется окно графического интерфейса. **MainApplication** в нем прописываются размеры и указывается путь к fxml файлу. **hello-view.fxml** -файл, в котором отрисовывается разметка главного окна. Мы не будем добавлять ее вручную, вместо этого будем использовать программу Scene Builder, которая выполняет эти действия автоматически.

1. **SalaryPayments** - лабораторная работа 1. Выдача зарплаты. **salaries.fxml**- разметка для лабораторной работы 1.
2. **Ciphers** – лабораторная работа 2. Фиксированный шифр и шифр Цезаря. **cipher.fxml**- разметка для лабораторной работы 2.
3. **QrCode** – лабораторная работа 3. Генератор QR кода. **qr-code.fxml**- разметка для лабораторной работы 3.
4. **AtmBank** – лабораторная работа 4. АТМ банк. **atm-bank.fxml**- разметка для лабораторной работы 4.
5. **Birds** - лабораторная работа 5. Применение паттернов к графическим элементам (пингвины и птицы). **birds.fxml**- разметка для лабораторной работы 5.
6. **module-info.java** – этот файл содержит информацию о модуле, включая его зависимости от других модулей и его собственные экспортируемые пакеты

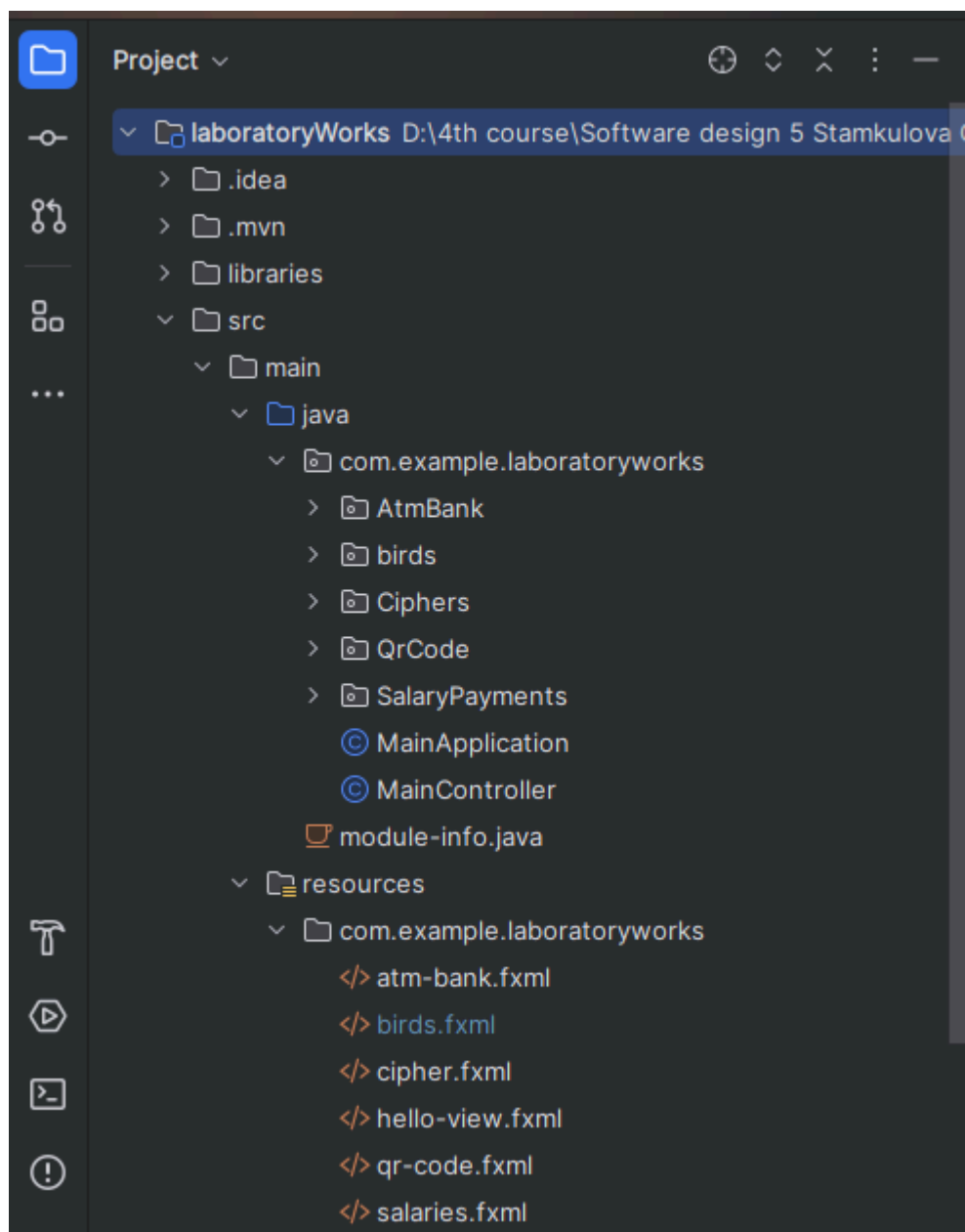


Рис. 11 Структура проекта

Открываем программу Scene Builder, как на рис. 12. Далее -> “Open project” (“Открыть проект”) -> найдите ваш проект и укажите путь к файлу “**hello-view.fxml**” проект.

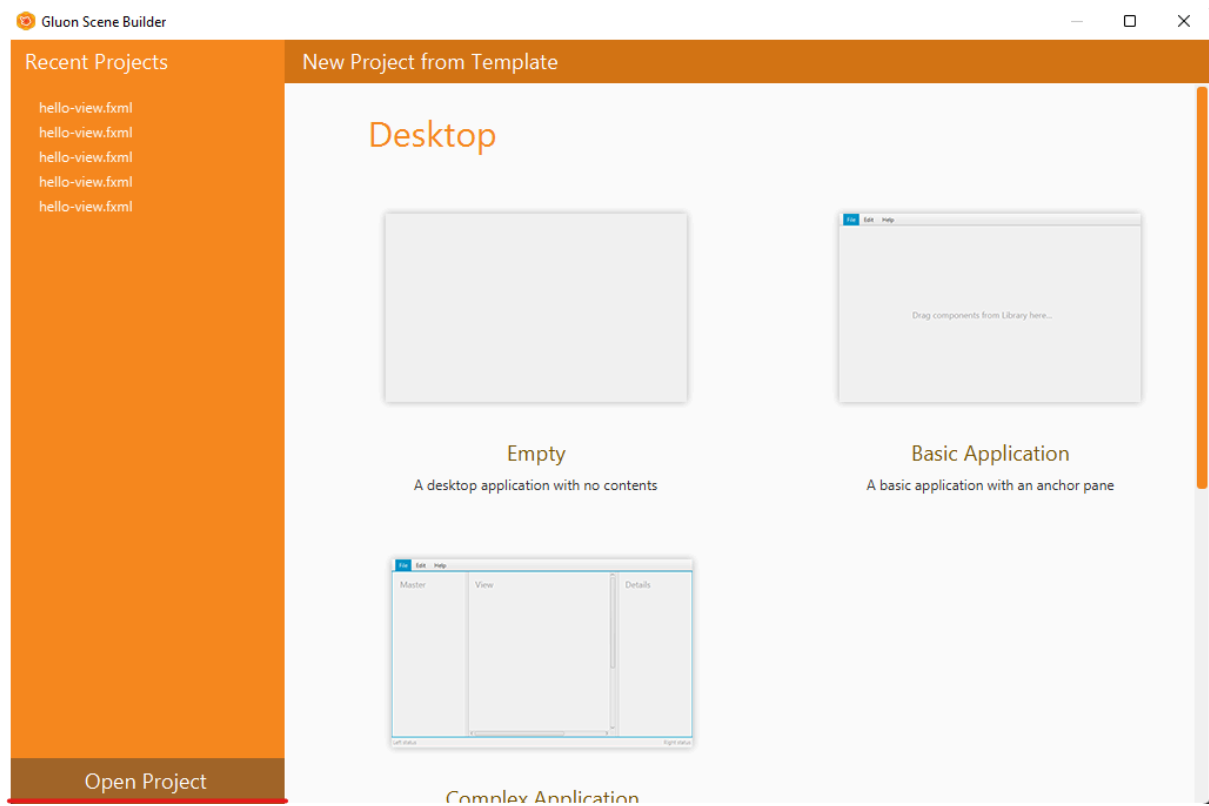


Рис. 12 Стартовое окно программы Scene Builder.

В обычном случае откроется небольшое окно с одной кнопкой. Но так как интерфейс был построен заранее, то он именно он появится автоматически, как на рис. 13 В Scene Builder есть несколько разделов: левая панель - для выбора элементов (TextField, Button, TableView, Pane), центр - результирующий интерфейс, справа - панель для декорирования элементов и управления их поведением.

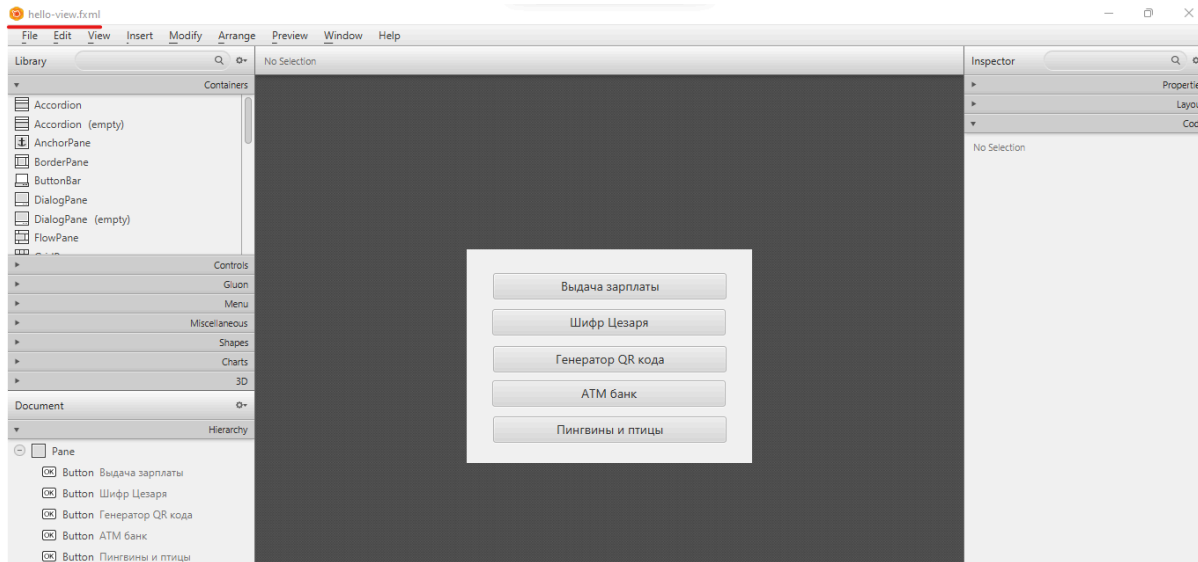


Рис. 13 Главное окно с открытым .fxml файлом

Выстраивайте интерфейс, используя левую боковую панель.

Также чтобы задать имена всем элементам интерфейса используйте правую панель, раздел “Code”. В нем необходимо указать имя элемента “fx:id”, действие “onAction”, как на рис. 14

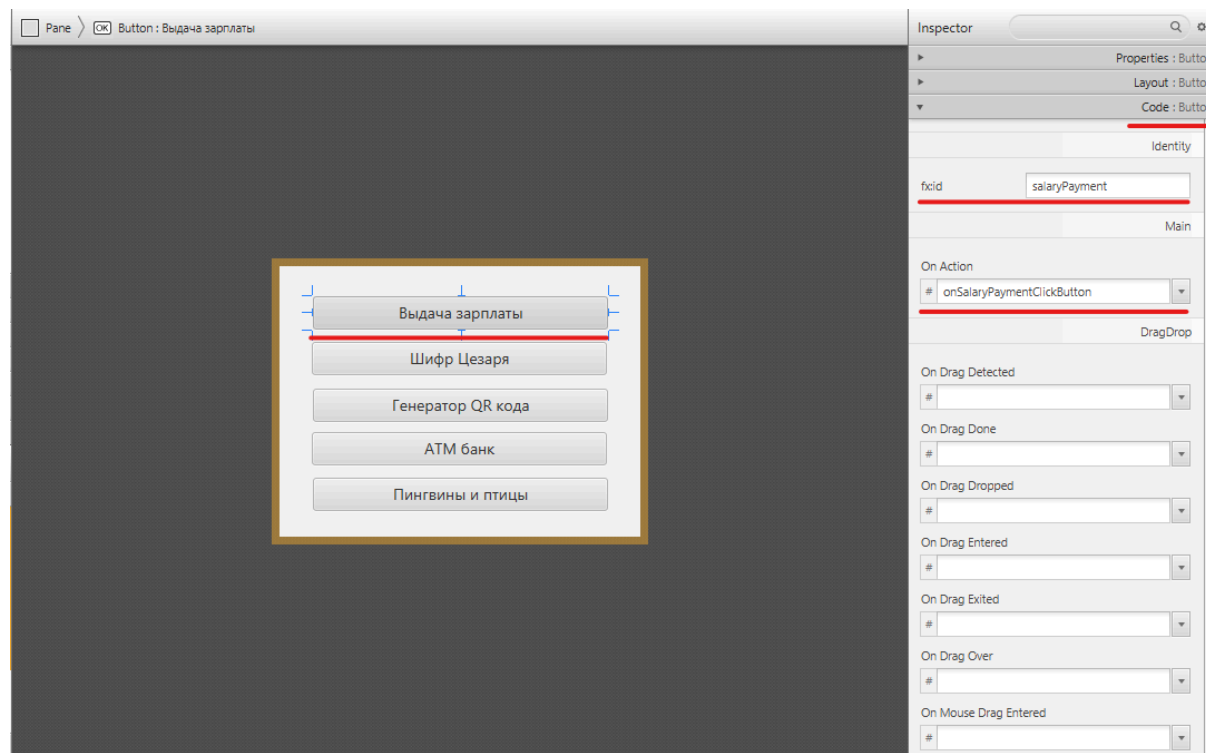


Рис. 14 Именованние элементов интерфейса

Название кнопки	fx:id	On Action
Выдача зарплаты	salaryPayment	onSalaryPaymentClickButton
Шифр Цезаря	ciphers	onCiphersClickButton
Генератор QR кода	qrCode	onQrCodeClickButton
АТМ банк	atmBank	onAtmBankClickButton
Пингвины и птицы	penguinsBirdsButton	onPenguinsBirdsClickButton

После создания элементов, не забудьте сохранить файл.

После того как вы построили интерфейс главного окна, необходимо перенести элементы интерфейса в проект на IntelliJ IDEA в файл **MainController.java**. Для этого в разделе “View” выбирайте “Show sample Controller skeleton”, то есть вы хотите получить все переменные, объявленные в этом интерфейсе (рис 15)

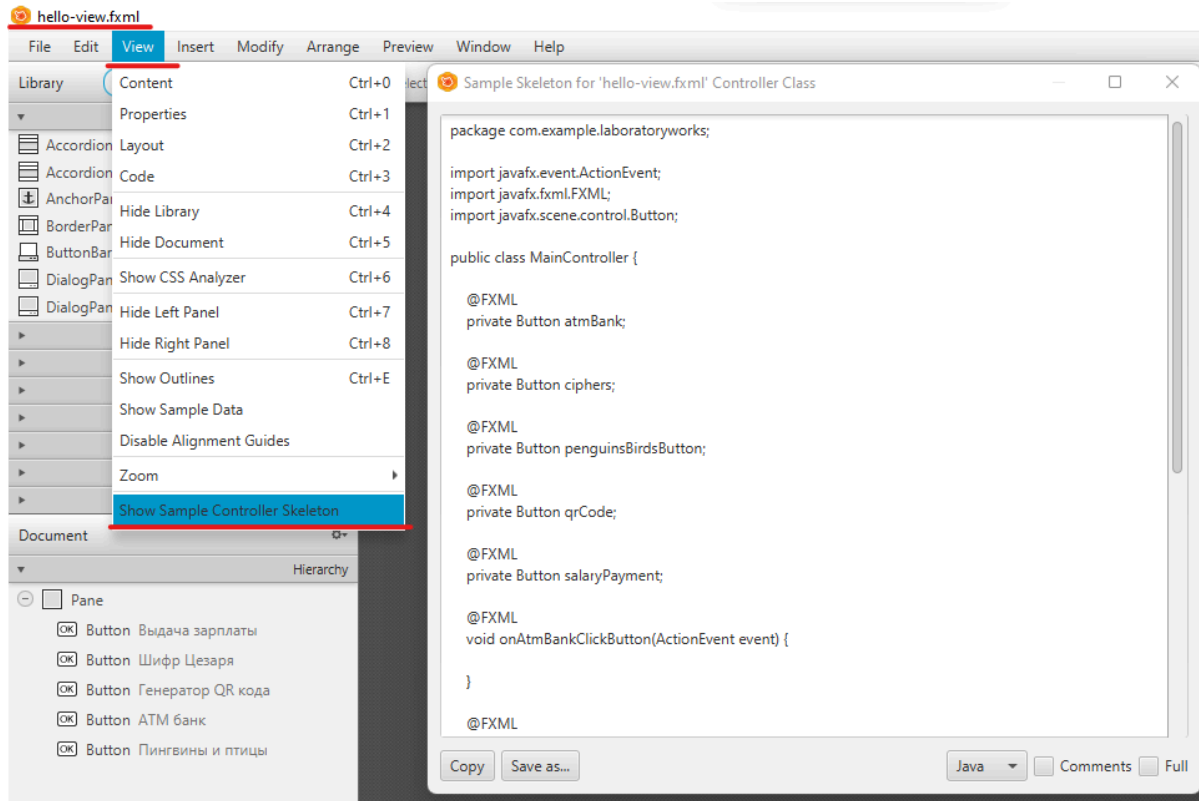


Рис. 15 Пример кода элементов интерфейса

Мы создали разметку для главного окна, теперь создадим разметку для лабораторной работы №1 - Зарплаты. Создайте salaries.fxml. Откройте этот файл через Scene Builder

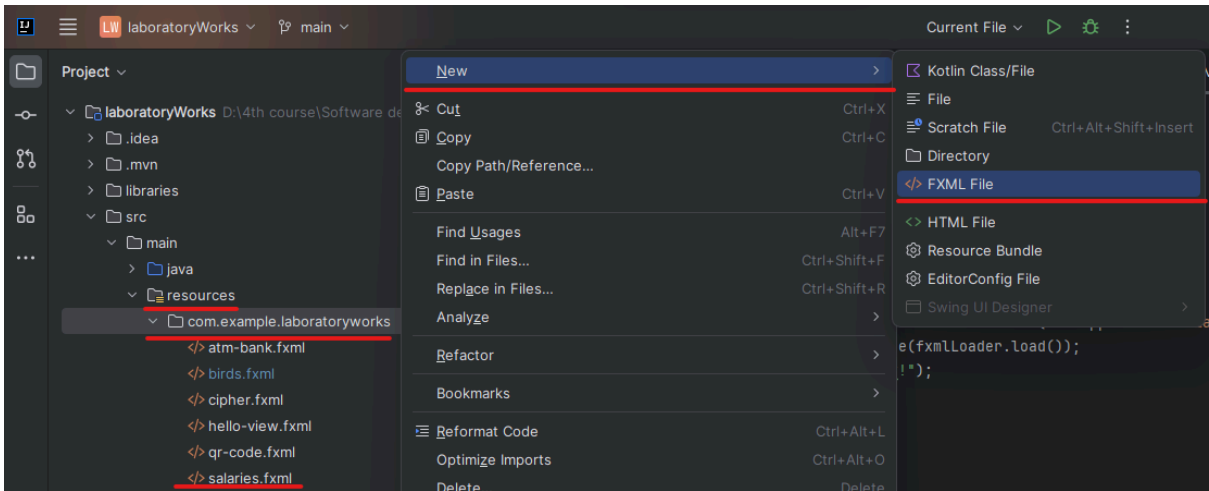


Рис. 16 Создание salaries.fxml

Далее создайте такую разметку:

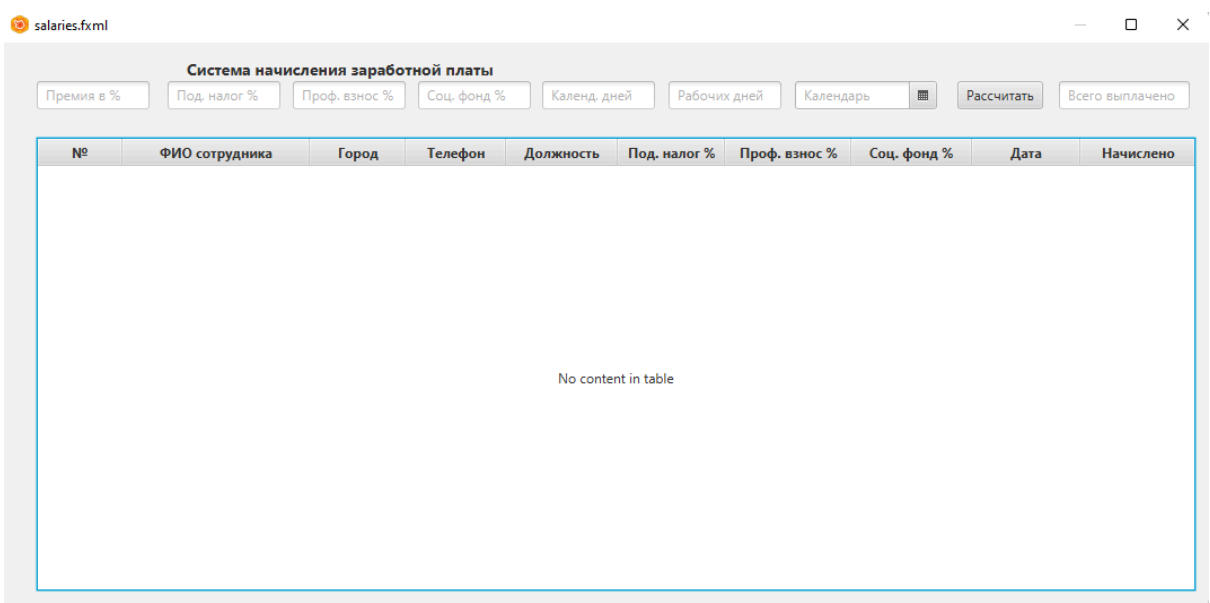


Рис. 17 разметка salaries.fxml

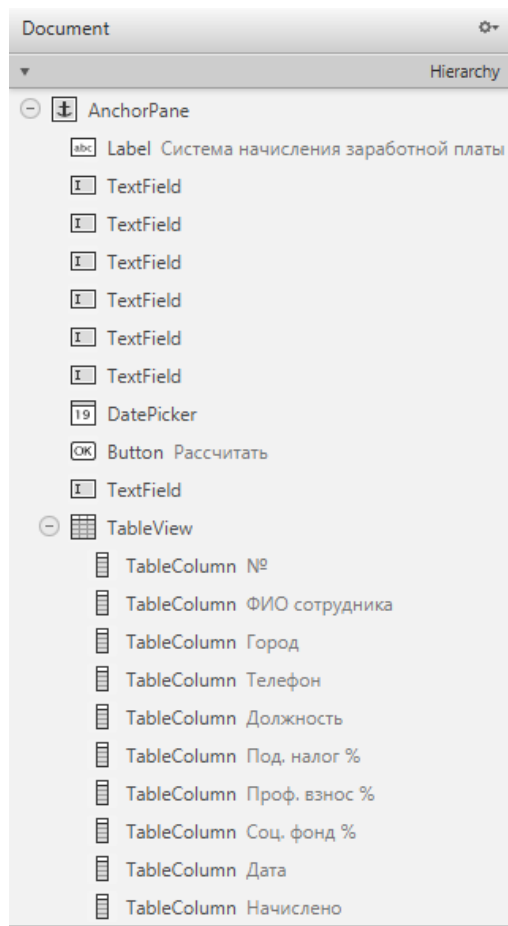
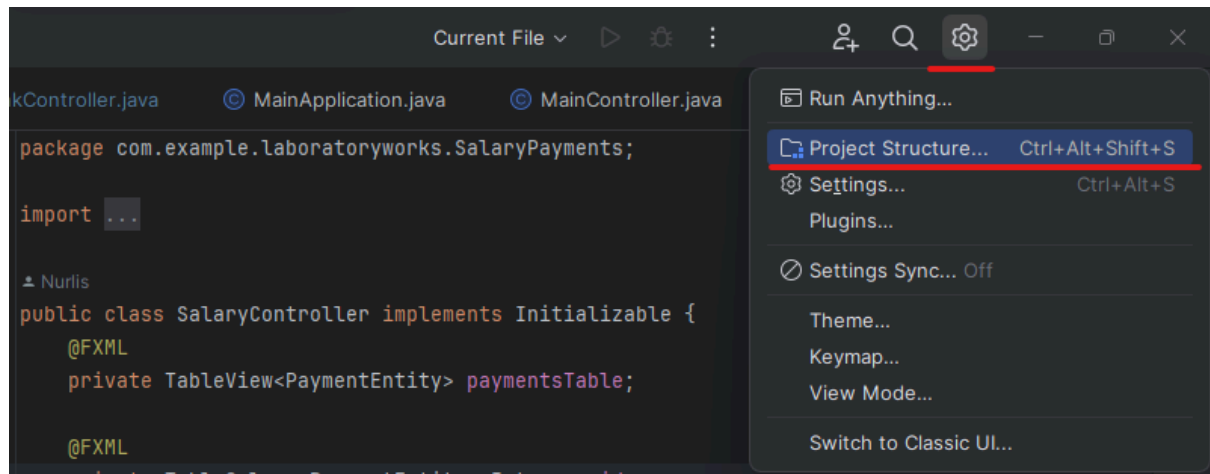


Рис. 18 документ salaries.fxml

Именованние элементов интерфейса		
Наименование элемента	fx:id	On Action
Премия в %	benefitMoneyField	
Под. налог %	profitTaxField	
Проф. взнос %	profTaxField	
Соц. фонд %	retirementTaxField	
Календ. дней	calendarDaysField	
Рабочих дней	workDaysField	
Календарь	dateField	
Рассчитать	countButton	countSalaries
Всего выплачено	totalPaiedField	
Таблица	paymentsTable	
№	id	
ФИО сотрудника	name	
Город	address	
Телефон	phone	
Должность	positionName	
Под. налог %	profit tax	
Проф. взнос %	prof tax	

Соц. Фонд %	retirement tax	
Дата	date	
Начислено	total	

Также для работы с базой данных необходимо подключить модуль **mysql connector** в проект. Для этого переходите в раздел “Project structure” (“Структура проекта”) => “Modules” => раздел “Зависимости” (“Dependencies”), а также **javafx-swing.jar**. В корне проекта создайте папку **libraries**, где будете хранить необходимые библиотеки.



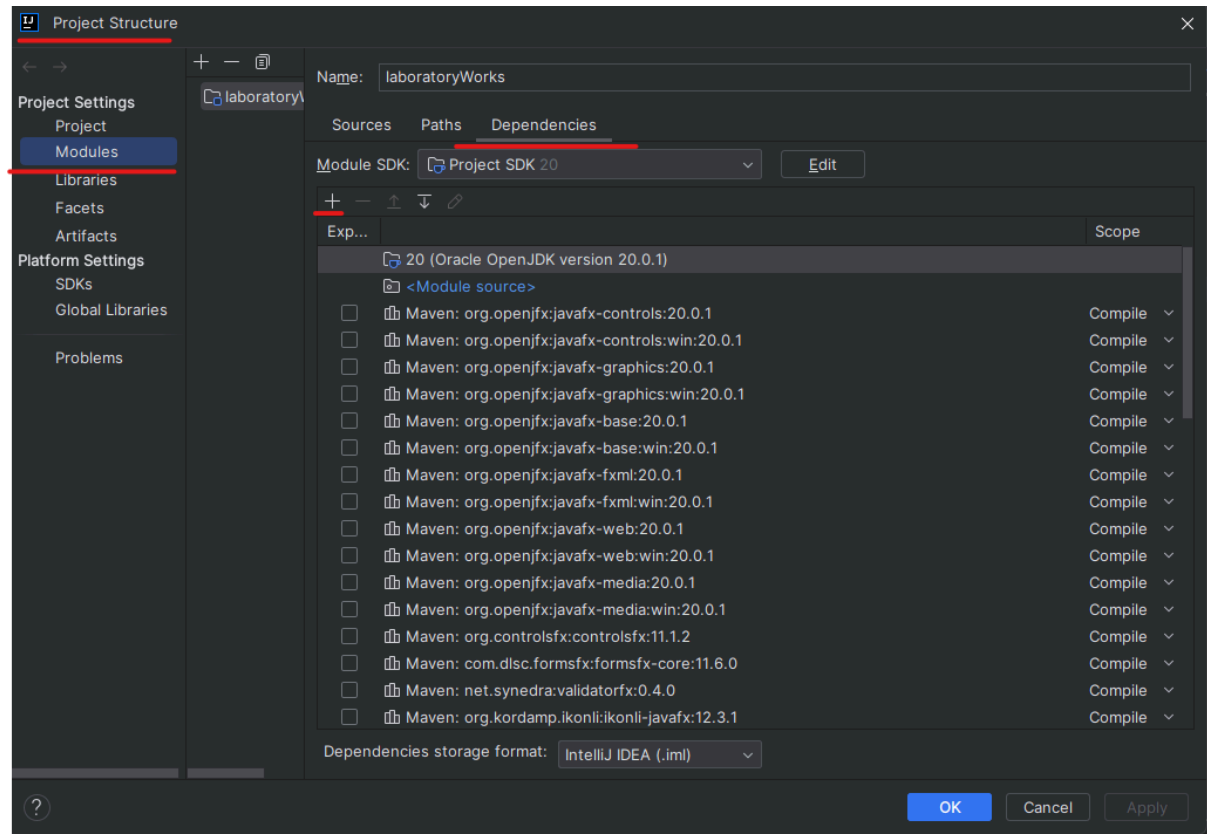
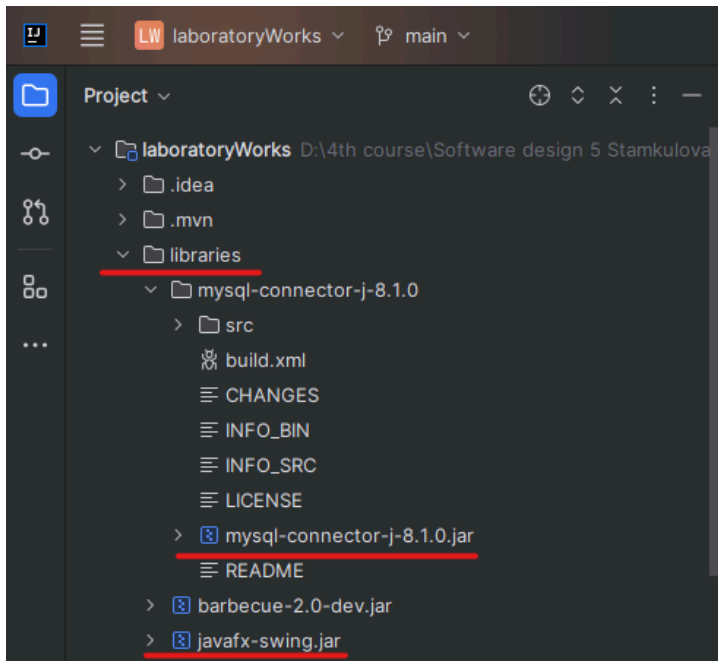


Рис. 19 Окно модулей проекта

Нажимаем “+”, выбираем пункт “JAR” (потому что в распакованном пакете mysql connector имеется именно .jar file), рис. 18

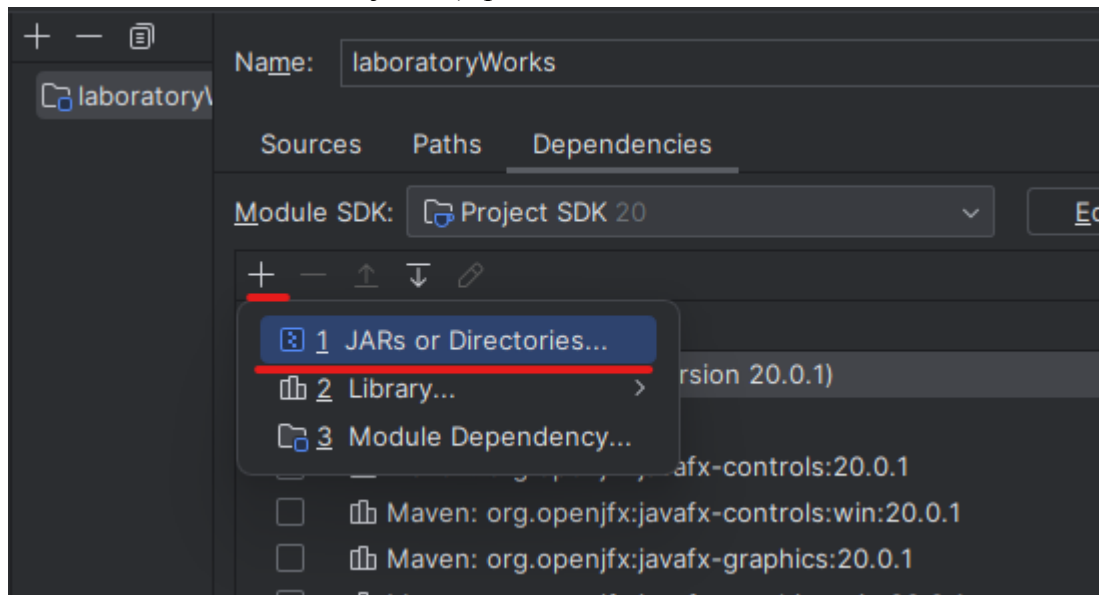


Рис. 20 Выбор типа импортируемого модуля

Далее указываем непосредственный путь к к mysql connector (лучше, если эта папка будет внутри вашего проекта), рис. 21

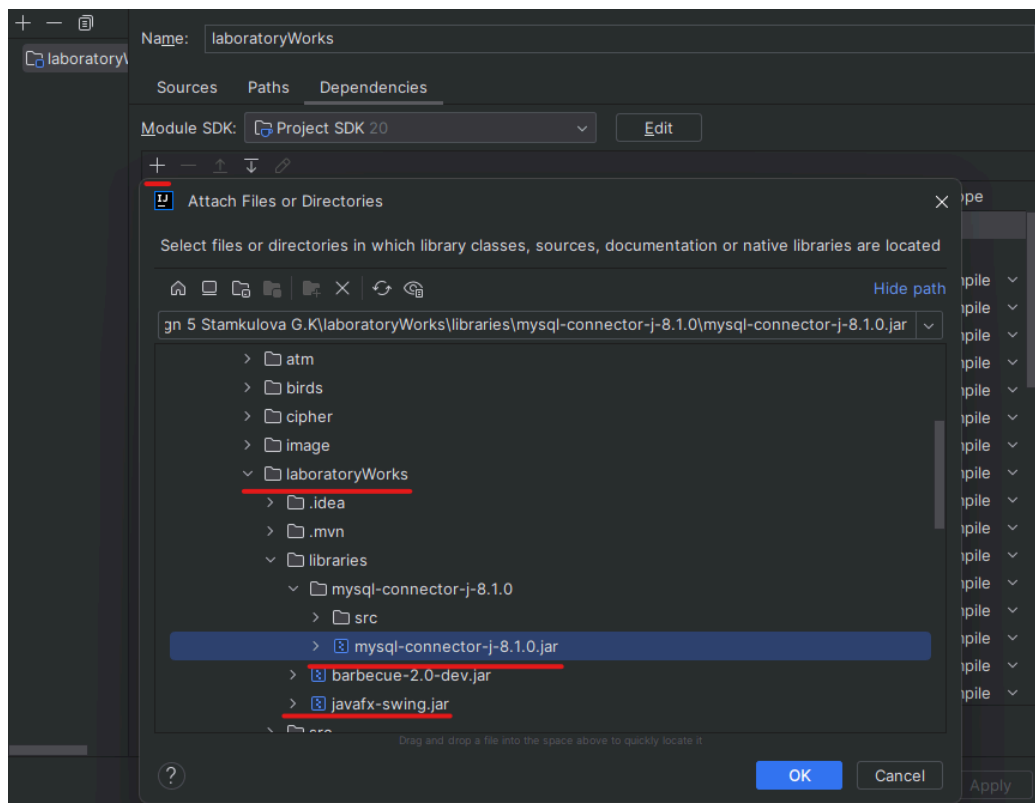


Рис. 21 Выбор .jar file

Далее необходимо настроить 4 расширения.

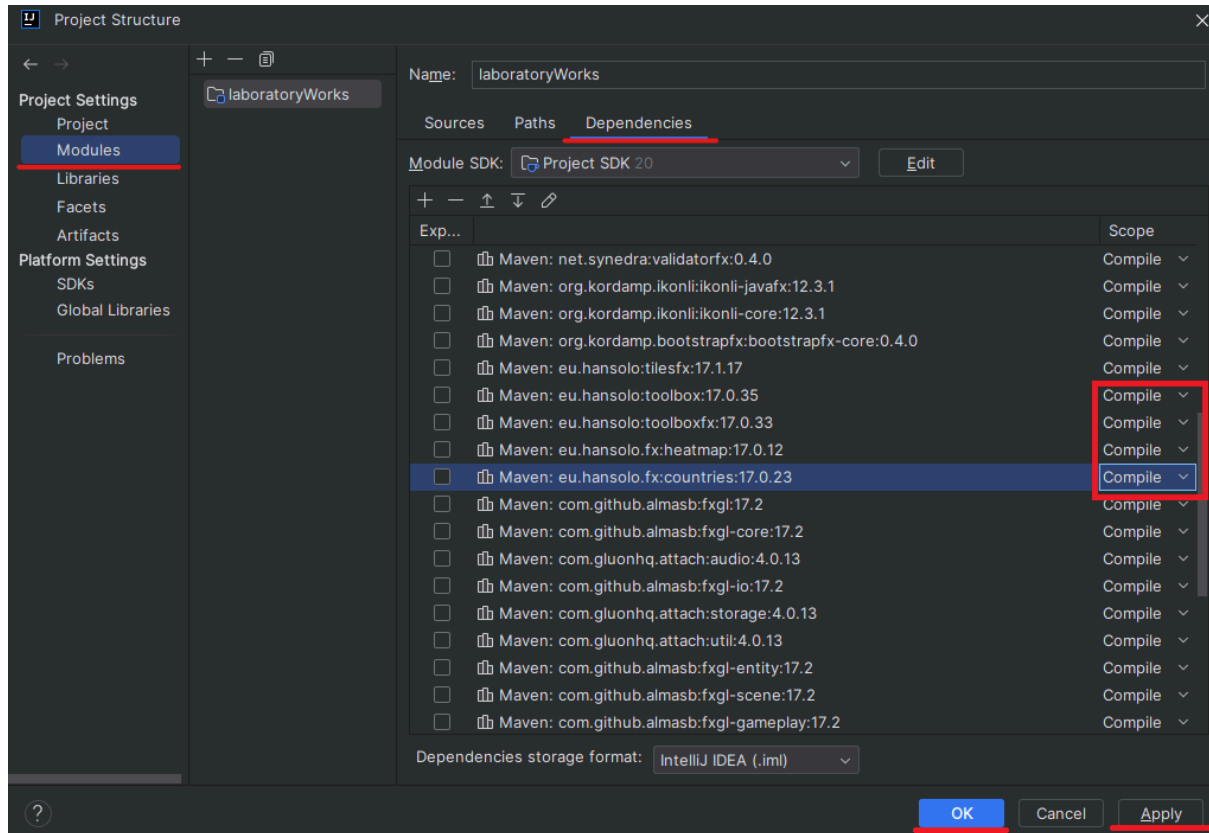


Рис. 22 Настройка расширений

Изначально у вас эти 4 расширения будут как **Runtime**, необходимо их установить как **Compile**.

Необходимо также добавить в файл **module-info.java**:

```
opens com.example.laboratoryworks.SalaryPayments to javafx.fxml;  
exports com.example.laboratoryworks.SalaryPayments;
```

exports com.example.laboratoryworks.SalaryPayments;

Эта директива означает, что модуль (или пакет) с именем **com.example.laboratoryworks.SalaryPayments** явно разрешает другим модулям доступ к своим классам и ресурсам, которые находятся в этом пакете. То есть, другие модули могут импортировать классы из этого пакета и использовать их.

opens com.example.laboratoryworks.SalaryPayments to javafx.fxml;

Эта директива связана с рефлексией и означает, что пакет **com.example.laboratoryworks.SalaryPayments** открывается для рефлексии (доступа к закрытым членам классов) для модуля **javafx.fxml**. Это позволяет модулю **javafx.fxml** получить доступ к закрытым членам классов в этом пакете

для выполнения динамических операций, таких как создание экземпляров классов и вызовов закрытых методов.

Рассмотрим также структуру БД. Имеется 3 таблицы: Positions - должности (рис. 23), Employees - работники (рис. 24), Payments - выплаты (рис. 26)

Table Name: positions Schema: salaries_db

Charset/Collation: utf8mb4 utf8mb4_0900 Engine: InnoDB

Comments:

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
id	INT	☒	☒	☐	☐	☐	☐	☒	☐	
name	VARCHAR(45)	☐	☒	☐	☐	☐	☐	☐	☐	
salary	DOUBLE	☐	☒	☐	☐	☐	☐	☐	☐	

Рис. 23 Таблица Positions

Table Name: employees Schema: salaries_db

Charset/Collation: utf8mb4 utf8mb4_0900 Engine: InnoDB

Comments:

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
id	INT	☒	☒	☐	☐	☐	☐	☒	☐	
name	VARCHAR(45)	☐	☒	☐	☐	☐	☐	☐	☐	NULL
address	VARCHAR(45)	☐	☒	☐	☐	☐	☐	☐	☐	NULL
phone	VARCHAR(45)	☐	☒	☐	☐	☐	☐	☐	☐	NULL
positionId	INT	☐	☒	☐	☐	☐	☐	☐	☐	NULL
age	VARCHAR(45)	☐	☒	☐	☐	☐	☐	☐	☐	NULL

Рис. 24

Таблица Employees

Table Name: employees Schema: salaries_db

Charset/Collation: utf8mb4 utf8mb4_0900_ai_cs Engine: InnoDB

Comments:

Foreign Key Name	Referenced Table	Column	Referenced Column
employees_positions_positions_id	salaries_db.positions	positionId	id

Foreign Key Options:

On Update: CASCADE

On Delete: CASCADE

☐ Skip in SQL generation

Рис. 25 Внешний ключ employees_positions_positions_id

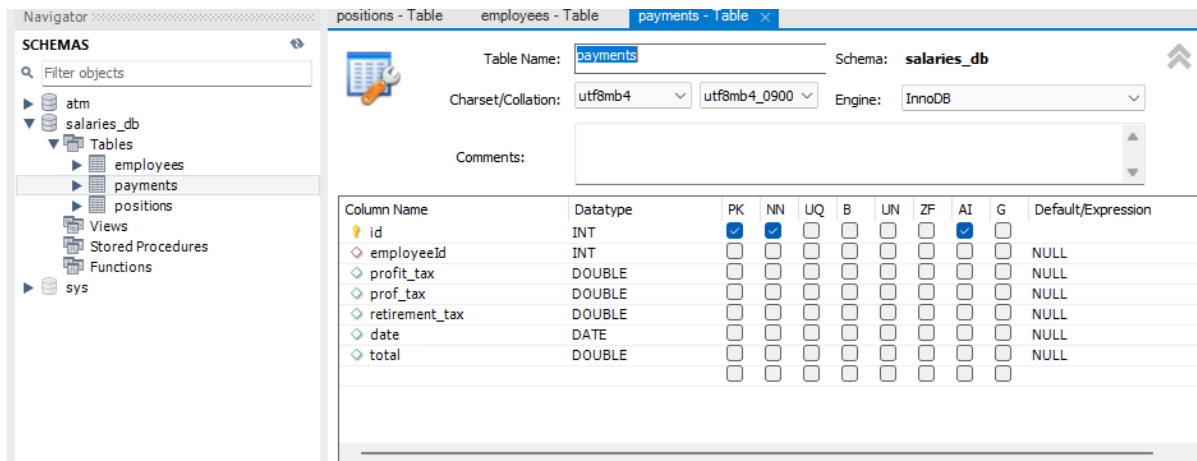


Рис. 26

Таблица Payments

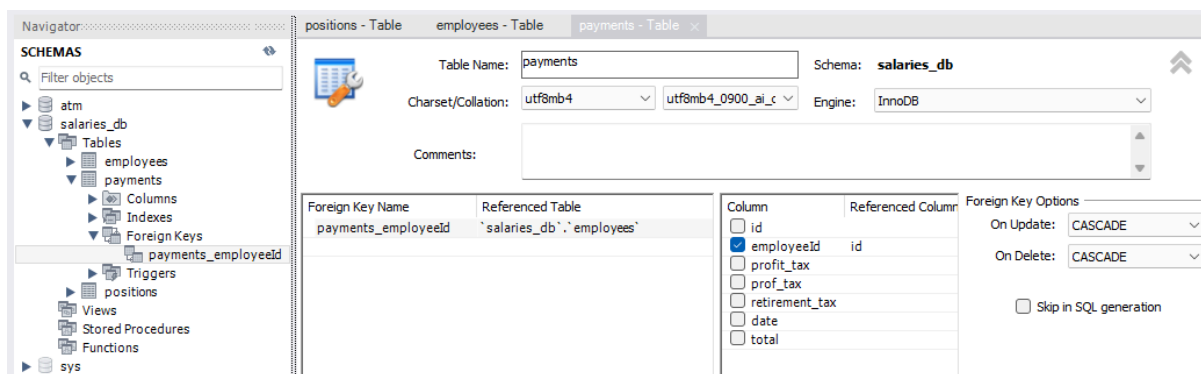


Рис. 27 Внешний ключ **payments_employeeId**

MainApplication.java:

```
package com.example.laboratoryworks;

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Scene;
import javafx.stage.Stage;

import java.io.IOException;

public class MainApplication extends Application {
    @Override
    public void start(Stage stage) throws IOException {
        FXMLLoader fxmlLoader = new FXMLLoader(MainApplication.class.getResource("hello-view.fxml"));
        Scene scene = new Scene(fxmlLoader.load());
        stage.setTitle("Привет!");
        stage.setScene(scene);

        stage.show();
    }

    public static void main(String[] args) {
        launch();
    }
}
```

MainController.java

```
package com.example.laboratoryworks;

import com.example.laboratoryworks.birds.BirdsController;
import javafx.event.ActionEvent;
import javafx.fxml.FXML;
import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.stage.Stage;

public class MainController {

    @FXML
    private Button ciphers;

    @FXML
    private Button salaryPayment;

    @FXML
    void onSalaryPaymentClickButton(ActionEvent event) {
        try {
            // Load the "salaries.fxml" file
            FXMLLoader loader = new FXMLLoader(getClass().getResource("salaries.fxml"));
            Parent root = loader.load();

            // Create a new stage and set the scene to "salaries.fxml"
            Stage stage = new Stage();
            stage.setTitle("Выдача зарплаты!");
            stage.setScene(new Scene(root));
            stage.show();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    @FXML
    public void onCiphersClickButton(ActionEvent actionEvent) {
        try {
            FXMLLoader loader = new FXMLLoader(getClass().getResource("cipher.fxml"));
            Parent root = loader.load();

            Stage stage = new Stage();
            stage.setTitle("Шифр Цезаря!");
            stage.setScene(new Scene(root));
            stage.show();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    public void onQRCodeClickButton(ActionEvent actionEvent) {
        try {
            FXMLLoader loader = new FXMLLoader(getClass().getResource("qr-code.fxml"));

```

```

        Parent root = loader.load();

        Stage stage = new Stage();
        stage.setTitle("QR код!");
        stage.setScene(new Scene(root));
        stage.show();
    } catch (Exception e) {
        e.printStackTrace();
    }
}

public void onAtmBankClickButton(ActionEvent actionEvent) {
    try {
        FXMLLoader loader = new FXMLLoader(getClass().getResource("atm-bank.fxml"));
        Parent root = loader.load();

        Stage stage = new Stage();
        stage.setTitle("АТМ банк!");
        stage.setScene(new Scene(root));
        stage.show();
    } catch (Exception e) {
        e.printStackTrace();
    }
}

private BirdsController birdsController; // Reference to BirdsController

@FXML
void onPenguinsBirdsClickButton(ActionEvent actionEvent) {
    try {
        FXMLLoader loader = new FXMLLoader(getClass().getResource("birds.fxml"));
        Parent root = loader.load();

        birdsController = loader.getController(); // Get the reference to BirdsController

        Stage stage = new Stage();
        stage.setTitle("Птицы!");
        stage.setScene(new Scene(root));
        stage.show();

        // Set the close request handler for the child window
        stage.setOnCloseRequest(event -> {
            if (birdsController != null) {
                birdsController.stopMediaPlayer(); // Call the stop method in BirdsController
            }
        });
    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

SalaryController.java

```
package com.example.laboratoryworks.SalaryPayments;

import com.example.laboratoryworks.SalaryPayments.Windows.ErrorWindow;
import javafx.collections.ObservableList;
import javafx.event.ActionEvent;
import javafx.fxml.FXML;
import javafx.fxml.Initializable;
import javafx.scene.control.*;
import javafx.scene.control.cell.PropertyValueFactory;

import java.net.URL;
import java.sql.Connection;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.time.LocalDate;
import java.util.ResourceBundle;

public class SalaryController implements Initializable {
    @FXML
    private TableView<PaymentEntity> paymentsTable;

    @FXML
    private TableColumn<PaymentEntity, Integer> id;

    @FXML
    private TableColumn<PaymentEntity, String> name;

    @FXML
    private TableColumn<PaymentEntity, String> address;

    @FXML
    private TableColumn<PaymentEntity, String> phone;

    @FXML
    private TableColumn<PaymentEntity, String> positionName;

    @FXML
    private TableColumn<PaymentEntity, Double> profit_tax;

    @FXML
    private TableColumn<PaymentEntity, Double> prof_tax;

    @FXML
    private TableColumn<PaymentEntity, Double> retirement_tax;

    @FXML
    private TableColumn<PaymentEntity, LocalDate> date;

    @FXML
    private TableColumn<PaymentEntity, Double> total;
```

```

@FXML
private TextField profitTaxField;

@FXML
private TextField profTaxField;

@FXML
private TextField retirementTaxField;

@FXML
private DatePicker dateField;

@FXML
private Button countButton;

@FXML
private TextField totalPaiedField;

@FXML
private TextField calendarDaysField;

@FXML
private TextField workDaysField;

@FXML
private TextField benefitMoneyField;

public void setInitialValues() throws SQLException {
    DatabaseConnection dbConnection = new DatabaseConnection();
    Connection connection = dbConnection.getDatabaseLink();
    ObservableList<PaymentEntity> payments = dbConnection.getPayments(connection);
    String totalPayments = dbConnection.countAllPayments(connection);
    if (totalPayments != null) {
        totalPaiedField.setText(totalPayments);
    } else {
        // Handle the case where the value is null, e.g., set a default value or show an error message.
    }
    paymentsTable.setItems(payments);
}

@Override
public void initialize(URL url, ResourceBundle resourceBundle) {
    id.setCellValueFactory(new PropertyValueFactory<>("id"));
    name.setCellValueFactory(new PropertyValueFactory<>("name"));
    address.setCellValueFactory(new PropertyValueFactory<>("address"));
    phone.setCellValueFactory(new PropertyValueFactory<>("phone"));
    positionName.setCellValueFactory(new PropertyValueFactory<>("positionName"));
    profit_tax.setCellValueFactory(new PropertyValueFactory<>("profit_tax"));
    prof_tax.setCellValueFactory(new PropertyValueFactory<>("prof_tax"));
    retirement_tax.setCellValueFactory(new PropertyValueFactory<>("retirement_tax"));
    date.setCellValueFactory(new PropertyValueFactory<>("date"));
    total.setCellValueFactory(new PropertyValueFactory<>("total"));

    try {
        setInitialValues();
    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

```

@FXML
void countSalaries(ActionEvent event) throws SQLException {
    if (Integer.parseInt(calendarDaysField.getText()) < Integer.parseInt(workDaysField.getText())) {
        new ErrorWindow().showWindow("Расчет зарплат", "Количество рабочих дней больше количества календарных дней. Измените ввод");
        return;
    }

    DatabaseConnection dbConnection = new DatabaseConnection();
    Connection connection = dbConnection.getDatabaseLink();
    ResultSet employeesResultSet = dbConnection.getEmployees(connection);
    dbConnection.createPayments(
        connection,
        employeesResultSet,
        Double.parseDouble(profitTaxField.getText()),
        Double.parseDouble(profTaxField.getText()),
        Double.parseDouble(retirementTaxField.getText()),
        Integer.parseInt(calendarDaysField.getText()),
        Integer.parseInt(workDaysField.getText()),
        Double.parseDouble(benefitMoneyField.getText()),
        dateField.getValue());

    ObservableList<PaymentEntity> payments = dbConnection.getPayments(connection);
    paymentsTable.setItems(payments);
    totalPaiedField.setText(dbConnection.countAllPayments(connection));
}
}

```

PaymentEntity.java

```
package com.example.laboratoryworks.SalaryPayments;

import java.time.LocalDate;

public class PaymentEntity {
    private int id;
    private String name;
    private String address;
    private String phone;
    private String positionName;
    private Double profit_tax;
    private Double prof_tax;
    private Double retirement_tax;
    private LocalDate date;
    private Double total;

    public PaymentEntity(int id, String name, String address, String phone, String positionName, Double
profit_tax, Double prof_tax, Double retirement_tax, LocalDate date, Double total) {
        this.id = id;
        this.name = name;
        this.address = address;
        this.phone = phone;
        this.positionName = positionName;
        this.profit_tax = profit_tax;
        this.prof_tax = prof_tax;
        this.retirement_tax = retirement_tax;
        this.date = date;
        this.total = total;
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public String getAddress() {
        return address;
    }

    public String getPhone() {
        return phone;
    }
}
```



```
public String getPositionName() {
    return positionName;
}

public Double getProfit_tax() {
    return profit_tax;
}

public Double getProf_tax() {
    return prof_tax;
}

public Double getRetirement_tax() {
    return retirement_tax;
}

public LocalDate getDate() {
    return date;
}

public Double getTotal() {
    return total;
}

public static Double countWorkedSalary(Double salary, Integer calendarDays, Integer workDays) {
    return salary / calendarDays * workDays;
}

public static Double countBenefitMoney(Double salary, Double benefitMoney) {
    return salary * benefitMoney / 100;
}

public static Double countTaxes(Double salary, Double bonus, Double profit_tax, Double prof_tax, Double
retirement_tax) {
    return ((salary + bonus) * (profit_tax + prof_tax + retirement_tax)) / 100;
}

public static Double countActualSalary(Double salary, Double bonus, Double taxes) {
    return (salary + bonus) - taxes;
}
}
```

DataBaseConnection.java

Здесь необходимо настроить подключение к своей БД!!!

```
package com.example.laboratoryworks.SalaryPayments;

import javafx.collections.FXCollections;
import javafx.collections.ObservableList;

import java.sql.*;
import java.text.DecimalFormat;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

public class DatabaseConnection {
    public Connection databaseLink;

    public Connection getDatabaseLink() {
        String dbName = "salaries_db";
        String dbNameUser = "root";
        String dbNamePass = "Nurlis2003";
        String dbNameUrl = "jdbc:mysql://localhost/" + dbName;
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            databaseLink = DriverManager.getConnection(dbNameUrl, dbNameUser, dbNamePass);
        } catch (Exception err) {
            err.printStackTrace();
        }
        return databaseLink;
    }

    public ObservableList<PaymentEntity> getPayments(Connection connection) throws SQLException {
        try {
            ObservableList<PaymentEntity> payments = FXCollections.observableArrayList();
            Statement statement = connection.createStatement();
            ResultSet resultSet = statement.executeQuery(" " +
                "SELECT payments.id, employees.name, employees.address, employees.phone, positions.name as " +
                "positionName, payments.profit_tax, payments.prof_tax, payments.retirement_tax, payments.date, payments.total " +
                "FROM payments " +
                "LEFT JOIN employees ON payments.employeeId = employees.id " +
                "LEFT JOIN positions ON employees.positionId = positions.id");

            while (resultSet.next()) {
```

```

        PaymentEntity payment = new PaymentEntity(
            Integer.parseInt(resultSet.getString("id")),
            resultSet.getString("name"),
            resultSet.getString("address"),
            resultSet.getString("phone"),
            resultSet.getString("positionName"),
            Double.parseDouble(resultSet.getString("profit_tax")),
            Double.parseDouble(resultSet.getString("prof_tax")),
            Double.parseDouble(resultSet.getString("retirement_tax")),
            LocalDate.parse(resultSet.getString("date"), DateTimeFormatter.ofPattern("yyyy-MM-dd")),
            Double.parseDouble(resultSet.getString("total")));
        payments.add(payment);
    }
    return payments;
} catch (Exception error) {
    error.printStackTrace();
    return null;
}

}

public ResultSet getEmployees(Connection connection) throws SQLException {
    Statement statement = connection.createStatement();
    ResultSet resultSet = statement.executeQuery("
        SELECT employees.id, positions.salary
        FROM employees
        LEFT JOIN positions ON employees.positionId = positions.id;");

    return resultSet;
}

public void createPayments(Connection connection, ResultSet employeesResultSet, Double profit_tax,
Double prof_tax, Double retirement_tax, Integer calendarDays, Integer workDays, Double benefitPercent,
LocalDate date) throws SQLException {
    while (employeesResultSet.next()) {
        Double dbSalary = Double.parseDouble(employeesResultSet.getString("salary"));
        Integer employeeId = Integer.parseInt(employeesResultSet.getString("id"));
        Double workedSalary = PaymentEntity.countWorkedSalary(dbSalary, calendarDays, workDays);
        Double benefitMoney = PaymentEntity.countBenefitMoney(workedSalary, benefitPercent);
        System.out.println("Benefit money: " + benefitMoney);
        Double taxes = PaymentEntity.countTaxes(workedSalary, benefitMoney, profit_tax, prof_tax,
retirement_tax);
        System.out.println("Taxes: " + taxes);
        Double actualSalary = PaymentEntity.countActualSalary(workedSalary, benefitMoney, taxes);
        System.out.println("Actual salary: " + actualSalary);

        try {
            Statement statement = connection.createStatement();
            statement.executeUpdate("INSERT INTO payments (employeeId, profit_tax, prof_tax, retirement_tax,
date, total) " +
                "VALUES(" + employeeId + ", " + profit_tax + ", " + prof_tax + ", " + retirement_tax + ", " + date + ",
"+actualSalary+)");
        } catch (SQLException err) {
            err.printStackTrace();
        }
    }
}

public String countAllPayments(Connection connection) throws SQLException {
    Statement statement = connection.createStatement();
    ResultSet resultSet = statement.executeQuery("SELECT SUM(payments.total) AS count FROM

```

```

payments");
    resultSet.next();
    return new DecimalFormat("#.##").format(Double.parseDouble(resultSet.getString("count")));
}
}

```

ErrorWindow.java

```

package com.example.laboratoryworks.SalaryPayments.Windows;

import javafx.scene.control.Alert;
import javafx.scene.control.ButtonType;

import java.util.Optional;

public class ErrorWindow {
    public void showWindow(String header, String message) {
        Alert alert = new Alert(Alert.AlertType.ERROR);
        alert.setTitle("Ошибка");
        alert.setHeaderText(header);
        alert.setContentText(message);
        Optional<ButtonType> result = alert.showAndWait();
    }
}

```

ConfirmationWindow.java

```

package com.example.laboratoryworks.SalaryPayments.Windows;

import javafx.scene.control.Alert;
import javafx.scene.control.ButtonType;

import java.util.Optional;

public class ConfirmationWindow {
    public Optional<ButtonType> showConfirmationWindow(String header, String message) {
        Alert alert = new Alert(Alert.AlertType.CONFIRMATION);
        alert.setTitle("Подтвердите действие");
        alert.setHeaderText(header);
        alert.setContentText(message);
        return alert.showAndWait();
    }
}

```

```
}  
}
```

AlertWindow.java

```
package com.example.laboratoryworks.SalaryPayments.Windows;  
  
import javafx.scene.control.Alert;  
import javafx.scene.control.ButtonType;  
  
import java.util.Optional;  
  
public class AlertWindow {  
    public void showWarningWindow(String header, String message) {  
        Alert alert = new Alert(Alert.AlertType.WARNING);  
        alert.setTitle("Предупреждение");  
        alert.setHeaderText(header);  
        alert.setContentText(message);  
        Optional<ButtonType> result = alert.showAndWait();  
    }  
  
    public void showErrorWindow(String header, String message) {  
        Alert alert = new Alert(Alert.AlertType.ERROR);  
        alert.setTitle("Ошибка");  
        alert.setHeaderText(header);  
        alert.setContentText(message);  
        Optional<ButtonType> result = alert.showAndWait();  
    }  
}
```

Полный код файла **module-info.java** выглядит следующим образом.

```
module com.example.laboratoryworks {  
    requires javafx.controls;  
    requires javafx.fxml;  
    requires javafx.web;  
  
    requires org.controlsfx.controls;  
    requires com.dlsc.formsfx;  
    requires net.synedra.validatorfx;  
    requires org.kordamp.ikonli.javafx;  
    requires org.kordamp.bootstrapfx.core;  
    requires eu.hansolo.tilesfx;  
    requires com.almasb.fxgl.all;  
    requires java.sql;  
    requires barbecue;  
  
    exports com.example.laboratoryworks;  
    exports com.example.laboratoryworks.SalaryPayments;  
    exports com.example.laboratoryworks.Ciphers;  
    exports com.example.laboratoryworks.QrCode;  
    exports com.example.laboratoryworks.AtmBank;  
    exports com.example.laboratoryworks.birds;  
    opens com.example.laboratoryworks to javafx.fxml;  
    opens com.example.laboratoryworks.SalaryPayments to javafx.fxml;  
    opens com.example.laboratoryworks.Ciphers to javafx.fxml;  
    opens com.example.laboratoryworks.QrCode to javafx.fxml;  
    opens com.example.laboratoryworks.AtmBank to javafx.fxml;  
}
```

```
opens com.example.laboratoryworks.birds to javafx.fxml;
```

Здесь перечислены модули и библиотеки, необходимые для всех лабораторных работ. Если возникают ошибки, следует удалить или закомментировать соответствующие записи. В зависимости от конкретной лабораторной работы, необходимо добавить нужные модули и библиотеки. Модули и библиотеки, выделенные **зеленым** цветом, обязательно должны быть включены для первой лабораторной работы.

В результате при запуске открывается окно, как на рис. 28

[illegible]

Рис. 29 Рабочая программа для вычисления зарплат